

# Universidad Nacional de General Sarmiento

Licenciatura en Sistemas

## Trabajo Práctico Final: Desarrollo de Videojuego 2D

Módulos de Implementación y Arquitectura de Software

**Asignatura:** Programación 1

**Integrantes del Equipo:** Calderón Gonzalo, Ayala Matias y Díaz Maximiliano

**Correos Electrónicos:** [gonzalocal127@gmail.com](mailto:gonzalocal127@gmail.com) - [mmattias0105@gmail.com](mailto:mmattias0105@gmail.com) - [Diazmaximilianonahuel33@gmail.com](mailto:Diazmaximilianonahuel33@gmail.com)

**Fecha de Entrega:** 9 de junio, 2026

## Introducción

El presente documento detalla el desarrollo e implementación de un videojuego bidimensional interactivo de plataformas y acción, construido sobre el lenguaje de programación **Java**. La infraestructura gráfica, el procesamiento de periféricos de entrada (teclado) y la sincronización del ciclo de actualización principal se gestionan mediante la biblioteca académica `Entorno` y su clase complementaria `Herramientas`.

El diseño del software se encuentra estrictamente fundamentado bajo el paradigma de la **Programación Orientada a Objetos (POO)**. La meta principal del juego consiste en guiar a un avatar ("Princesa") a través de un escenario generado de manera aleatoria. El primer nivel introduce dinámicas de desplazamiento de pantalla lateral (*scrolling*), recolección de recursos e interacciones con amenazas, culminando en una estructura de transición ("Castillo"). El segundo nivel altera el flujo al fijar la cámara en un escenario cerrado donde se produce un combate singular contra una entidad jefa ("Jefe"), la cual posee distintos comportamientos.

## Descripción de Clases e Implementación

A continuación, se describen exhaustivamente cada una de las trece clases que conforman el sistema, detallando sus campos de instancia, sus métodos característicos y su funcionalidad en el ecosistema del juego.

### 1. Clase: `Juego`

**Función:** Funciona como el punto de entrada principal (*Main*) de la aplicación. Es la encargada de instanciar el lienzo gráfico de `Entorno`, inicializar el orquestador de escenarios y centralizar la máquina de estados del juego mediante el método cíclico `tick()`.

| Variables de Instancia                  | Descripción Técnica                                                                                                              |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>private Entorno entorno</code>    | Referencia al lienzo del motor gráfico para el procesamiento visual y de periféricos.                                            |
| <code>private Niveles niveles</code>    | Instancia encargada de coordinar la lógica física y visual del gameplay de los mapas.                                            |
| <code>private String estadoJuego</code> | Cadena que determina el estado de la aplicación ( <code>"JUGANDO"</code> , <code>"GAME_OVER"</code> , <code>"VICTORIA"</code> ). |

| Métodos Principales                       | Descripción Funcional                                                                                                                           |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public Juego()</code>               | Constructor. Instancia el objeto <code>Entorno</code> , crea el controlador de niveles e inicia la ejecución.                                   |
| <code>public void tick()</code>           | Bucle principal del juego ejecutado de forma continua. Distribuye el flujo de renderizado y lógica según el valor de <code>estadoJuego</code> . |
| <code>public static void main(...)</code> | Método de entrada estándar que da inicio al ciclo de vida del programa.                                                                         |

## 2. Clase: `Niveles`

**Función:** Funciona como la clase orquestadora del gameplay. Controla de forma específica las fases del juego, conteniendo las referencias a los elementos activos, inicializando los mapas desde cero y administrando el desplazamiento de la cámara en zonas con *scroll*.

| Variables de Instancia                                  | Descripción Técnica                                                              |
|---------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>private Entorno entorno</code>                    | Enlace al lienzo gráfico compartido por las entidades.                           |
| <code>private Princesa princesa</code>                  | Referencia al personaje controlado por el usuario.                               |
| <code>private GestionadorPlataformas plataformas</code> | Controlador del conjunto de superficies transitables.                            |
| <code>private GestionadorEnemigos enemigos</code>       | Controlador del lote de amenazas del Nivel 1.                                    |
| <code>private Jefe jefe</code>                          | Instancia del antagonista final del Nivel 2.                                     |
| <code>private Proyectil proyectil</code>                | Referencia a la entidad disparable y recolectable por el jugador en el entorno.  |
| <code>private Image fondoLv11, imagenVictoria</code>    | Recursos visuales cargados en memoria para fondos de pantalla.                   |
| <code>private double camaraX, camaraRecorrido</code>    | Coordenadas e indicadores de desplazamiento para el efecto de <i>scrolling</i> . |
| <code>private int nivel</code>                          | Indicador entero del nivel en curso (1 o 2).                                     |

| Métodos Principales                                     | Descripción Funcional                                                                                                                                       |
|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public void<br/>inicializarNivel1()</code>        | Construye el mapa inicial, posicionando el piso básico, las islas flotantes, enemigos y el castillo de meta.                                                |
| <code>public void<br/>inicializarNivel2()</code>        | Reinicia el entorno físico. Re-instanciando las colecciones, posiciona a la princesa en coordenadas fijas e inicializa al Jefe.                             |
| <code>public void<br/>actualizarGameplay()</code>       | Determina qué nivel se encuentra activo y deriva el flujo al método ejecutor correspondiente ( <code>ejecutarNivel1</code> o <code>ejecutarNivel2</code> ). |
| <code>private void<br/>actualizarCamara(...)</code>     | Calcula el desplazamiento incremental de la pantalla basándose en la posición horizontal de la princesa.                                                    |
| <code>private void<br/>aparecerProyectilRandom()</code> | Mecánica probabilística interna del Nivel 1 que introduce proyectiles aleatorios desde el suelo.                                                            |

### 3. Clase: `Princesa`

**Función:** Encapsula los atributos y comportamientos del personaje principal. Controla su física posicional (vectores de salto y gravedad), colisiones con límites de pantalla, consumo de salud, animaciones y estados de invulnerabilidad temporal tras recibir daño.

| Variables de Instancia                          | Descripción Técnica                                               |
|-------------------------------------------------|-------------------------------------------------------------------|
| <code>private double x, y</code>                | Coordenadas cartesianas del centro del personaje.                 |
| <code>private double ancho,<br/>alto</code>     | Dimensiones espaciales para delimitar la caja de colisión (AABB). |
| <code>private int vidas</code>                  | Contador de intentos de juego restantes del personaje.            |
| <code>private boolean<br/>esInvulnerable</code> | Flag temporal que inhabilita la recepción de daño consecutivo.    |

| Métodos Principales                        | Descripción Funcional                                                                                 |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>public void moverPrincesa()</code>   | Aplica las fuerzas físicas de gravedad e interpreta las teclas de dirección para alterar la posición. |
| <code>public void dibujarPrincesa()</code> | Muestra el gráfico correspondiente al personaje en sus coordenadas relativas.                         |
| <code>public void perderVida()</code>      | Resta una unidad al contador de salud y activa el estado de invulnerabilidad temporal.                |
| <code>public boolean estaMuerta()</code>   | Retorna verdadero si el contador de vidas es menor o igual a cero.                                    |

#### 4. Clase: `Plataforma`

**Función:** Representa un bloque geométrico individual y rígido dentro del entorno del juego, actuando como zona de soporte para las entidades dinámicas.

| Variables de Instancia                        | Descripción Técnica                                                                                    |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>private double x, y, ancho, alto</code> | Valores que definen la posición y el volumen del bloque en el plano bidimensional.                     |
| <code>private boolean esPozo</code>           | Flag lógico que determina si la instancia representa un pozo(ver <code>gestionadorPlataformas</code> ) |

| Métodos Principales                                             | Descripción Funcional                                                                                                                                                                                                       |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public void dibujar(double camaraX)</code>                | Dibuja el bloque aplicando el desfase horizontal provocado por el movimiento de la cámara.                                                                                                                                  |
| <code>public void ColisionConPrincesa(Princesa princesa)</code> | Implementa el algoritmo de colisión AABB (Axis-Aligned Bounding Box). Calcula el solapamiento (overlap) en ambos ejes y reubica la posición de la princesa para evitar que atraviese el bloque o se trabe en sus laterales. |
| <code>public void colisionaConVida(VidaExtra vida)</code>       | Evalúa de manera bidimensional la intersección de áreas entre la plataforma y el ítem de vida extra que cae, anulando la velocidad vertical de este último al tocar la superficie.                                          |

#### 5. Clase: `GestionadorPlataformas`

**Función:** Encapsula los arreglos de objetos tipo `Plataforma`. Se encarga de la generación estructural de la topografía del nivel (suelos y plataformas suspendidas) y centraliza la resolución de colisiones físicas de soporte.

| Variables de Instancia                      | Descripción Técnica                                                 |
|---------------------------------------------|---------------------------------------------------------------------|
| <code>private Plataforma[]<br/>piso</code>  | Colección indexada de bloques que componen el suelo firme inferior. |
| <code>private Plataforma[]<br/>islas</code> | Arreglo de estructuras suspendidas a diferentes alturas del plano.  |

| Métodos Principales                                                 | Descripción Funcional                                                                                                |
|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>public void crearPiso(int<br/>cant, Entorno e)</code>         | Instancia secuencialmente los bloques base para cubrir la extensión del Nivel 1, creando pozos de manera secuencial. |
| <code>public void<br/>colisionesPlataformas(...)</code>             | Comprueba la interacción de intersección de la princesa e ítems con las plataformas e islas.                         |
| <code>public void<br/>dibujarPlataformas(double<br/>camaraX)</code> | Itera sobre los arreglos activos invocando sus métodos de renderizado bajo el desfase de cámara correspondiente.     |

## 6. Clase: [Enemigo](#)

**Función:** Define el comportamiento de un oponente genérico móvil del Nivel 1. Gestiona su desplazamiento autónomo alternando direcciones al alcanzar extremos lógicos de patrullaje.

| Variables de Instancia                          | Descripción Técnica                                           |
|-------------------------------------------------|---------------------------------------------------------------|
| <code>private double x, y,<br/>velocidad</code> | Estado cinemático de la entidad y rapidez de traslación.      |
| <code>private int direccion</code>              | Multiplicador de sentido (-1 para izquierda, 1 para derecha). |

| Métodos Principales                                  | Descripción Funcional                                                                                |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>public void mover()</code>                     | Actualiza la coordenada horizontal sumando el producto de la velocidad y la dirección de patrullaje. |
| <code>public void<br/>dibujar(double camaraX)</code> | Renderiza el elemento gráfico en su posición corregida respecto a la cámara.                         |

## 7. Clase: `GestionadorEnemigos`

**Función:** Administra masivamente el ciclo de vida de los enemigos normales en el mapa, automatizando el renderizado secuencial, la actualización de sus movimientos y la remoción por colisiones letales.

| Variables de Instancia                         | Descripción Técnica                                                  |
|------------------------------------------------|----------------------------------------------------------------------|
| <pre>private Enemigo[]<br/>listaEnemigos</pre> | Estructura contenedora fija con los enemigos desplegados en el mapa. |

| Métodos Principales                                          | Descripción Funcional                                                                                       |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <pre>public void<br/>inicializarEnemigos(int<br/>cant)</pre> | Instancia un lote controlado de entidades hostiles en posiciones distribuidas del mapa.                     |
| <pre>public boolean<br/>actualizarEnemigos(...)</pre>        | Itera el conjunto procesando colisiones dañinas contra la princesa o interacciones con proyectiles activos. |

## 8. Clase: `Jefe`

**Función:** Modela al antagonista final del Nivel 2. Implementa una cantidad sustancial de puntos de resistencia y rutinas cíclicas dedicadas a la emisión de ráfagas de proyectiles dirigidos hacia el usuario.

| Variables de Instancia                         | Descripción Técnica                                                 |
|------------------------------------------------|---------------------------------------------------------------------|
| <pre>private double x, y</pre>                 | Ubicación espacial fija o flotante del jefe en el nivel de combate. |
| <pre>private int vida<br/>[ ]</pre>            | Atributos de estado que cuantifican la salud de la entidad.         |
| <pre>private JefeProyectil[]<br/>ataques</pre> | Arreglo dinámico/fijo que contiene las ráfagas balísticas emitidas. |

| Métodos Principales                                          | Descripción Funcional                                                                                 |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <pre>public void<br/>iniciarAtaque1(Princesa p)</pre>        | Calcula vectores e instancia proyectiles que orbitan la entidad                                       |
| <pre>public void<br/>actualizarAtaques(Princesa p)</pre>     | Actualiza la posición y procesa impactos contra el jugador, en base al ataque actual.                 |
| <pre>public boolean<br/>detectarColisionProyectil(...)</pre> | Verifica si un proyectil del protagonista colisiona con la entidad, bajando su vida en caso positivo. |

## 9. Clase: `JefeProyectil`

**Función:** Representa los proyectiles cinemáticos especializados disparados por el Jefe final, provistos de trayectorias vectoriales orientadas a interceptar a la princesa.

| Variables de Instancia                         | Descripción Técnica                                                                 |
|------------------------------------------------|-------------------------------------------------------------------------------------|
| <pre>private double x, y,<br/>velX, velY</pre> | Estado posicional y componentes vectoriales de velocidad en ambos ejes cartesianos. |

| Métodos Principales                                           | Descripción Funcional                                                                                                                                            |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>public void desplazar()</pre>                            | Modifica de forma lineal las coordenadas del proyectil basándose en sus componentes vectoriales internos.                                                        |
| <pre>moverDireccionPrincesa()</pre>                           | Actualiza de manera lineal las coordenadas cartesianas agregando el producto de las componentes vectoriales vx y vy por el escalar velocidad.                    |
| <pre>calcularAngulo(double<br/>centroX, double centroY)</pre> | Traduce coordenadas polares a cartesianas mediante funciones trigonométricas (cos y sin) para ubicar al proyectil en una órbita circular alrededor de un centro. |
| <pre>seSalioDelMapa(Entorno<br/>entorno)</pre>                | Evalúa si el proyectil ha abandonado los márgenes útiles del lienzo para habilitar su remoción de memoria.                                                       |
| <pre>colisionaCon(Princesa<br/>princesa)</pre>                | Ejecuta un test de solapamiento de cajas delimitadoras (AABB) entre el proyectil y la princesa, retornando un indicador lógico si se produce un impacto dañino.  |

## 10. Clase: `Proyectil`

**Función:** Gestiona los proyectiles generales que interactúan de forma física en las pantallas de juego, sirviendo tanto de obstáculo como de elemento interactivo mecánico para vulnerar la posición del jefe o enemigos.

| Variables de Instancia                    | Descripción Técnica                                                  |
|-------------------------------------------|----------------------------------------------------------------------|
| <pre>private double x, y,<br/>radio</pre> | Datos de localización y dimensiones circulares de colisión espacial. |

| Métodos Principales | Descripción Funcional |
|---------------------|-----------------------|
|---------------------|-----------------------|



|                                        |                                                                                                                                          |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>public boolean disparo(...)</pre> | Efectúa el desplazamiento cinemático de la entidad y retorna un flag lógico si el objeto abandona el área de visión útil de la pantalla. |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|

## 11. Clase: `GestionadorDeItems`

**Función:** Ejerce el control sobre las entidades coleccionables del mapa, regulando su aparición sobre las plataformas y su remoción de la memoria al ser recolectadas.

| Variables de Instancia                   | Descripción Técnica                                                              |
|------------------------------------------|----------------------------------------------------------------------------------|
| <pre>private VidaExtra[] itemsVida</pre> | Colección que almacena los recursos de bonificación de salud activos en el mapa. |

| Métodos Principales                         | Descripción Funcional                                                                                                        |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <pre>public void actualizarItems(...)</pre> | Actualiza el renderizado de los consumibles y valida si se produce contacto de proximidad con la princesa para su absorción. |

## 12. Clase: `VidaExtra`

**Función:** Representa un elemento interactivo estático en el entorno que funciona como un *power-up* de salud, otorgando un intento adicional al jugador al ser consumido.

| Variables de Instancia                      | Descripción Técnica                                                              |
|---------------------------------------------|----------------------------------------------------------------------------------|
| <pre>private double x, y, ancho, alto</pre> | Ubicación y delimitador rectangular para verificar la colisión por solapamiento. |

| Métodos Principales                              | Descripción Funcional                                           |
|--------------------------------------------------|-----------------------------------------------------------------|
| <pre>public void aplicarEfecto(Princesa p)</pre> | Incrementa el contador interno de vidas del avatar del jugador. |

## 13. Clase: `Castillo`

**Función:** Modela la estructura que marca el final del recorrido del Nivel 1. Actúa como hito físico de meta estática.

| Variables de Instancia            | Descripción Técnica                                                                    |
|-----------------------------------|----------------------------------------------------------------------------------------|
| <code>private double x, y</code>  | Coordenadas fijas donde se renderiza la estructura al final de la geografía del nivel. |
| <code>private Image imagen</code> | Recurso visual de la estructura arquitectónica.                                        |

| Métodos Principales                                     | Descripción Funcional                                                                                                                    |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>public boolean verificarVictoria(Princesa p)</pre> | Calcula mediante intersección de áreas si las cajas de colisión del avatar y de la estructura se superponen, validando el fin del nivel. |

## Problemas Encontrados, Decisiones y Soluciones

*Durante el desarrollo de la aplicación se presentaron desafíos técnicos significativos asociados con la arquitectura de software y la gestión de recursos de hardware en Java. A continuación se exponen las tres problemáticas críticas y el proceso de resolución adoptado:*

### 1. Problema 1: Acoplamiento de Estados Globales e Infracción del Principio de Responsabilidad Única (SRP)

*Descripción:* Originalmente, la clase `Niveles` administraba pantallas globales ajenas al juego activo (como la pantalla de `gameOver()` y `pantallaVictoria()`), provocando que controlara el ciclo vital

completo de la aplicación e hiper-trofiara sus responsabilidades.

*Decisión y Solución:* Se extrajeron dichos métodos hacia la clase conductora principal (`Juego` / Main). Se redefinió a `Niveles` para abocarse exclusivamente al control del gameplay de las pantallas activas, exponiendo métodos de consulta lógica (*getters*) tales como `princesaEstaMuerta()` y `getNivelActual()`. De este modo, la clase principal actúa como una máquina de estados limpia que conmuta de forma óptima el renderizado general.

### 2. Problema 2: Rigidez en la construcción de mapas dinámicos

*Descripción:* Definir las coordenadas absolutas x de cada plataforma dentro de su propio constructor volvía el diseño de niveles un proceso rígido y propenso a errores matemáticos al momento de encadenar superficies.

*Decisión y Solución:* Se optó por inicializar `this.x = 0` en el constructor de la entidad individual. Se delegó la responsabilidad del posicionamiento horizontal y el espaciado métrico al bucle iterativo de `GestionadorPlataformas`. Esto dota a la clase `Plataforma` de una alta cohesión y reutilización, permitiendo acoplarla dinámicamente a cualquier geografía de nivel sin modificar su código interno.

### 3. Generacion de islas en base al suelo

*Descripción:* Al generar las islas por arriba, muchas veces, por la aleatoriedad de los pozos, estas se pasaban del mapa o se quedaban cortas

.

*Decisión y Solución:* Se optó por tomar el valor x del ultimo piso generado, y calcular cuantas islas serian necesarias antes de inicializarlas.

## Implementación (Código Fuente Ilustrativo)

A continuación, se adjunta el fragmento de la clase `Proyectil`, en la que se evidencia la implementación de un proyectil funcional, basado en cálculos vectoriales y chequeos de colisión.

```
package juego;

import java.awt.Color;
import entorno.Entorno;

//Clase
public class Proyectil {
    private double x;
    private double y;
    private double ancho;
    private double alto;
    private double velocidad;

    // Componentes de velocidad para permitir cualquier diagonal/dirección
    private double vx;
    private double vy;

    private boolean disparado;
    private boolean equipado; // Evita el bug de quedarse flotando en el aire

    //Constructor
    public Proyectil(double x, double y) {
        this.ancho = 30;
        this.alto = 15;
        this.velocidad = 12; // Velocidad Óptima para las diagonales
        this.disparado = false;
        this.equipado = false;
        this.vx = 0;
        this.vy = 0;
        this.x = x;
        this.y = y;
    }

    //Metodos
    //Metodo1
    public void dibujar(Entorno entorno) {
        double angulo = 0;
        if (disparado) {
            angulo = Math.atan2(vy, vx);
        }
        entorno.dibujarRectangulo(this.x, this.y, this.ancho, this.alto, angulo, Color.RED);
    }

    //Metodo2
    public void actualizar(Princesa princesa) {
        if (!disparado) {
            // SI NO SE DISPARÓ: Sigue a la princesa
            this.x = princesa.getX();
            this.y = princesa.getY() - (princesa.getAlto() / 2);
        } else {
            // SI YA SE DISPARÓ: Se mueve de forma autónoma en la diagonal calculada
            this.x += this.vx;
            this.y += this.vy;
        }
    }

    //Metodo3
    // Calcula el vector de velocidad apuntando directamente al mouse (360 grados)
    public void disparar(int mouseX, int mouseY) {
        if (!this.disparado) {
            this.disparado = true;

            double deltaX = mouseX - this.x;
```

```

        double deltaY = mouseY - this.y;

        // Calculamos la distancia total (hipotenusa)
        double distancia = Math.sqrt(deltaX * deltaX + deltaY * deltaY);

        // Control de seguridad para evitar división por cero
        if (distancia == 0) {
            distancia = 1;
            deltaX = 1;
        }

        // Descomposición vectorial de la velocidad
        this.vx = (deltaX / distancia) * this.velocidad;
        this.vy = (deltaY / distancia) * this.velocidad;
    }

    //Metodo4 Devuelve true mientras el objeto siga dentro de los limites del mapa
    public boolean actualizarProyectil(Princesa princesa, Entorno entorno, double camaraX){
        // 1. Dibujarse en la pantalla
        this.dibujar(entorno);

        // 2. Manejar el agarre y el seguimiento a la princesa
        if (!this.disparado) {
            if (!this.equipado) {
                // SI ESTÁ EN EL SUELO: Se mueve con el desplazamiento de la cámara
                this.x -= camaraX;

                // Caja de colisión para agarrarlo del suelo por primera vez
                boolean tocando = (princesa.getX() - princesa.getAncho()/2 < this.x +
this.ancho/2 &&
                                princesa.getX() + princesa.getAncho()/2 > this.x -
this.ancho/2 &&
                                princesa.getY() - princesa.getAlto()/2 < this.y +
this.alto/2 &&
                                princesa.getY() + princesa.getAlto()/2 > this.y -
this.alto/2);
                if (tocando) {
                    this.equipado = true; // Se equipa y no se suelta más
                }
            }

            if (this.equipado) {
                this.actualizar(princesa);
            }
        } else {
            this.actualizar(princesa);
        }

        // 3. DETECTAR EL DISPARO CON EL CLICK DEL MOUSE
        if (entorno.sePresionoBoton(entorno.BOTON_IZQUIERDO) && this.equipado &&
!this.disparado) {
            this.disparar(entorno.mouseX(), entorno.mouseY());
        }

        // 4. Verificar si se salió del mapa (por los costados, arriba o abajo)
        if (this.seSalióDelMapa(entorno)) {
            return false;
        }
        return true;
    }

    //Metodo5
    public boolean seSalióDelMapa(Entorno entorno) {
        return (this.x > entorno.ancho() + 50 || this.x < -50 || this.y > entorno.alto() + 50
|| this.y < -50);
    }

    //Getters y Setters
    public boolean isDisparado() {
        return disparado;
    }

```

```
(...)  
}
```

## Conclusiones

---

El desarrollo de este proyecto permitió consolidar de forma práctica conceptos de arquitectura de software y diseño de sistemas bajo el paradigma de orientación a objetos. La modularización del código mediante clases "Gestionadoras" demostró ser una solución de diseño sumamente eficiente para desacoplar la lógica de comportamiento individual de las entidades (como las físicas de la princesa o las rutinas de patrullaje de un enemigo) del control macro estructural del mapa.

Asimismo, el proceso de refactorización de la clase de niveles hacia la clase principal puso de manifiesto el valor del principio de responsabilidad única en sistemas interactivos en tiempo real. Al descargar a la clase constructora de mapas de la gestión de pantallas globales independientes, no solo se incrementó la legibilidad general del código fuente, sino que se preparó la arquitectura interna de la aplicación para admitir futuras ampliaciones de escalabilidad de manera directa y segura (como la adición de pantallas de menú de opciones o nuevos niveles).

Finalmente, la comprensión y el aprovechamiento de los mecanismos nativos del lenguaje Java, en particular la administración dinámica de memoria efectuada por el *Garbage Collector* a través del proceso de des-referenciación (re-instanciación con la palabra clave `new`), evidenció que un diseño arquitectónico limpio influye directamente de forma positiva tanto en el control de errores de tipo nulo como en el rendimiento de los recursos del sistema de cómputo.